

Reviewer Pack — Invariant Generator Count Bounds SOS Refutation Degree for S...

Entry 6972f4aaf7c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 08:00:32 UTC

Invariant Generator Count Bounds SOS Refutation Degree for Symmetric CSPs

Entry ID: 6972f4aaf7c4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 08:00:32 UTC

1. Conjecture

Field A (mathematical branch): Invariant Theory **Field B** (complexity object): SOS Refutation Degree for CSPs

Statement:

For a CSP instance with symmetry group G , the SOS refutation degree is at most the number of invariant generators of its constraint polynomials under G . Formally, if P is a polynomial ideal defining the CSP's feasible solutions, then $\deg_SOS(P) \leq |\text{InvGen}(P, G)|$.

Rationale (proposer's reasoning):

Symmetries in CSPs often reduce the effective complexity by collapsing orbits of variables. Invariant theory provides tools to decompose constraint polynomials into symmetry-invariant components, which may directly limit the SOS hierarchy's required degree via orbit-stabilizer principles.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 798ed89b3c584e34

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    result = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented = [A[i] + [b[i]] for i in range(m)]
    for j in range(n):
        pivot_row = max(range(j, m), key=lambda x: abs(augmented[x][j]))
        augmented[j], augmented[pivot_row] = augmented[pivot_row], augmented[j]
        for i in range(j + 1, m):
            factor = augmented[i][j] / augmented[j][j]
            for k in range(n + 1):
                augmented[i][k] -= factor * augmented[j][k]
    x = [0] * n
    for j in range(n - 1, -1, -1):
        x[j] = augmented[j][-1]
        for i in range(j):
            augmented[i][-1] -= augmented[i][j] * x[j]
    return x

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n = 30
G = [[1, -1], [-1, 1]] #  $Z_2^n$  symmetry group for 2 variables

def is_invariant(poly, g):
    for i in range(n):
        if poly[i] != poly[g[0][i]]:
            return False
    return True

def generate_3sat_instance():
    clauses = []
    for _ in range(10 * n):
        literals = [random.randint(0, 1) for _ in range(n)]
        clause = [l if random.choice([True, False]) else -l for l in literals]
        clauses.append(clause)
    return clauses

def compute_sos_refutation_degree(clauses):
    # Placeholder for actual SOS refutation degree computation
    return len(clauses) # Simplified for testing purposes

def compute_invariant_generators(poly, G):
    m = len(poly)
    n = len(G[0])
    A = [[0] * (m + n) for _ in range(m)]
    b = [1] * m
    for i in range(m):
        A[i][i] = 1
    for g in G:
        for j in range(n):
            A[j][-1] += poly[g[0][j]]
    x = gaussian_elimination(A, b)
    return sum(abs(x[i]) for i in range(m))

clauses = generate_3sat_instance()
sos_refutation_degree = compute_sos_refutation_degree(clauses)
invariant_generators = 0

for poly in [1 if random.choice([True, False]) else -1 for _ in range(n)]:
    if is_invariant(poly, G):
        invariant_generators += 1

metric_name = "SOS Refutation Degree"
metric_value = sos_refutation_degree
instances_tested = n
conjecture_holds = sos_refutation_degree <= invariant_generators
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) + [53, 67, 71, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_lab6691.py", line 117, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_lab6691.py", line 94, in run_trial

if is_invariant(poly, G):

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_lab6691.py", line 60, in is_invariant

if poly[i] != poly[g[0][i]]:

~~~~^^

TypeError: 'int' object is not subscriptable

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed due to TypeError in subscripting integer object, preventing data collection  
| next: Fix the is\_invariant function to handle polynomial representations correctly

## 11. Audit log (LLM calls)

**Total LLM calls:** 8

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 111295       |
| 2 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 72947        |
| 3 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 24170        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20637        |
| 5 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 12651        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13438        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13135        |
| 8 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 17974        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 286247 ms total latency. Provider mix: {'ollama\_remote': 8}

(full prompt+response transcripts available in *research/audit/6972f4aaf7c4.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/6972f4aaf7c4.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/6972f4aaf7c4.tar.gz* (if generated)